Chapitre 3 : Les structures de contrôle en Java

Par Robert DIASSÉ

Introduction

Les structures de contrôle permettent de **définir le flux d'exécution** d'un programme.
En d'autres termes, elles permettent de choisir **quelles instructions seront exécutées** et **combien de fois**.

En Java, on distingue principalement trois familles de structures de contrôle :

1. **Les structures conditionnelles** → permettent de prendre des décisions.
2. **Les structures itératives (boucles)** → permettent de répéter des instructions.
3. **Les structures de contrôle de rupture** → permettent d'interrompre ou de continuer une boucle.

3.1 Les structures conditionnelles

3.1.1 `if` simple

```
import java.util.Scanner;

public class Controls{
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Entrez votre âge : ");
        int age = scanner.nextInt();

        if (age >= 18) {
            System.out.println("Vous êtes majeur.");
        }

        scanner.close();
    }
}
```

Explication :

- `if (age >= 18)` : la condition est vérifiée (ici, vrai car `age = 20`).
- Si la condition est vraie, on exécute l'instruction `System.out.println(...)`.
- Sinon, rien ne se passe.

Sortie attendue :

```
Vous êtes majeur.
```

3.1.2 `if ... else`

```java
import java.util.Scanner;

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Entrez votre âge : ");
    int age = scanner.nextInt();

    if (age >= 18) {
        System.out.println("Vous êtes majeur.");
    } else {
        System.out.println("Vous êtes mineur.");
    }

    scanner.close();
}
```

Explication :

- Si `age >= 18`, on affiche "Vous êtes majeur".
- Sinon (si ici `age = 15`), on affiche "Vous êtes mineur".

3.1.3 `if ... else if ... else`

```java
import java.util.Scanner;

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Entrez votre note : ");
    int note = scanner.nextInt();

    if (note >= 16) {
        System.out.println("Très bien");
    } else if (note >= 12) {
        System.out.println("Assez bien");
    } else if (note >= 10) {
```

```
        System.out.println("Passable");
    } else {
        System.out.println("Échec");
    }

    scanner.close();
}
```

Explication :

- Plusieurs conditions sont testées les unes après les autres.
- Si Ici `note = 14` → on tombe dans le cas `"Assez bien"`.

3.1.4 `switch`

```
import java.util.Scanner;

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Entrez un jour de la semaine (1-7) : ");
    int jour = scanner.nextInt();

    switch (jour) {
        case 1: System.out.println("Lundi"); break;
        case 2: System.out.println("Mardi"); break;
        case 3: System.out.println("Mercredi"); break;
        case 4: System.out.println("Jeudi"); break;
        case 5: System.out.println("Vendredi"); break;
        case 6: System.out.println("Samedi"); break;
        case 7: System.out.println("Dimanche"); break;
        default: System.out.println("Jour inconnu");
    }

    scanner.close();
}
```

Explication :

- `switch (jour)` compare `jour` avec chaque `case`.
- Ici `jour = 3` → on exécute `System.out.println("Mercredi");`.
- `break` empêche l'exécution des autres `case`.

3.2 Les structures itératives (boucles)

3.2.1 La boucle `for`

```java
import java.util.Scanner;

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Entrez un nombre de répétitions : ");
    int n = scanner.nextInt();

    for (int i = 1; i <= n; i++) {
        System.out.println("Compteur : " + i);
    }

    scanner.close();
}
```

Explication :

- `int i = 1` → initialisation.
- `i <= n` → condition de poursuite.
- `i++` → incrémentation à chaque tour.
- La boucle s'exécute 5 fois.

Sortie attendue :

```
Compteur : 1
Compteur : 2
.
.
.
Compteur : n
```

3.2.2 La boucle `while`

```java
import java.util.Scanner;

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Entrez un nombre positif : ");
    int i = scanner.nextInt();

    while (i < 0) {
        System.out.print("Erreur ! Entrez un nombre positif : ");
        i = scanner.nextInt();
    }

    System.out.println("Vous avez saisi : " + i);
    scanner.close();
```

```
}
```

Explication :

- La condition est vérifiée **avant chaque exécution**.
- Tant que `i < 0`, la boucle continue.

3.2.3 La boucle `do ... while`

```java
public class DoWhileLoop {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println("Compteur : " + i);
            i++;
        } while (i <= 5);
    }
}
```

Explication :

- La boucle s'exécute **au moins une fois**, même si la condition est fausse.
- Ici, on affiche les nombres de 1 à 5.

Sortie attendue :

```
Compteur : 1
Compteur : 2
Compteur : 3
Compteur : 4
Compteur : 5
```

3.3 Les structures de contrôle de rupture

3.3.1 `break`

```java
public class BreakExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                break; // on arrête la boucle
            }
            System.out.println("i = " + i);
        }
```

```
    }
}
```

Sortie attendue :

```
i = 1
i = 2
i = 3
i = 4
```

3.3.2 `continue`

```java
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // saute le tour où i = 3
            }
            System.out.println("i = " + i);
        }
    }
}
```

Sortie attendue :

```
i = 1
i = 2
i = 4
i = 5
```

Voici une proposition de conclusion pour le Chapitre 3, adaptée à ton style pédagogique et pour des étudiants en développement d'application mobile :

Conclusion

Les structures de contrôle constituent **le cœur de la programmation Java**. Elles permettent de :

1. **Prendre des décisions** grâce aux structures conditionnelles (`if`, `if...else`, `switch`),
2. **Répéter des instructions** avec les boucles (`for`, `while`, `do...while`),
3. **Gérer le flux d'exécution** à l'intérieur des boucles grâce à `break` et `continue`.

Maîtriser ces structures est indispensable pour :

- écrire des programmes corrects et lisibles,
- gérer la logique des applications mobiles (saisie utilisateur, validations, traitements répétitifs),